

BLOOD BANK MANAGEMENT SYSTEM

Prepared in partial fulfilment of the Summer Internship Program on AWS

AT



Under the guidance of
Mrs. Sumana Bethala, APSSDC

Submitted by

Yakama Venkata Keerthan Varma, 23BQ1A12F1, varmakeerthan9@gmail.com

Tiruveedula Naga Satish, 23BQ1A12D3, nagasathish06@gmail.com

Shaik Mouyin Imran, 23BQ1A12B6, imranimran39400@gmail.com

Sathvik Vemula, 23BQ1A12E8, sathvikvemula99@gmail.com

Pudi Shiva shankar, 23BQ1A1299, shivapudi06@gmail.com

ACKNOWLEDGEMENT

I would like to express my heartfelt gratitude to all those who have contributed to the successful completion of my summer internship project at Andhra Pradesh Skill Development Corporation (APSSDC). This opportunity has been an enriching and transformative experience for me, and I am truly thankful for the support, guidance, and encouragement I have received along the way.

First and foremost, I extend my sincere regards to Mrs Sumana Bethala and Mr J LovaBabu, my supervisors and mentors, for providing me with valuable insights, constant guidance, and unwavering support throughout the internship. Their expertise and encouragement have been instrumental in shaping the direction of this project.

I want to thank the entire team at Andhra Pradesh Skill Development Corporation (APSSDC) for fostering a collaborative and innovative environment. The camaraderie, knowledge sharing, and feedback I received from my colleagues significantly contributed to the development and success of this project.

In conclusion, I am honoured to have been a part of this internship program, and I look forward to leveraging the skills and knowledge gained to contribute positively to future endeavours.

Thank you.

Sincerely,

Yakama Venkata Keerthan Varma, 23BQ1A12F1, varmakeerthan9@gmail.com

Tiruveedula Naga Satish, 23BQ1A12D3, nagasathish06@gmail.com

Shaik Mouyin Imran, 23BQ1A12B6, imranimran39400@gmail.com

Sathvik Vemula, 23BQ1A12E8, sathvikvemula99@gmail.com

Pudi Shiva shankar, 23BQ1A1299, shivapudi06@gmail.com

ABSTRACT

The Blood Bank Management System is a full-stack web application designed to centralize donor records, blood bank information, blood inventory, blood requests, and blood issue records. Conventional blood management processes can depend on fragmented registers, spreadsheets, or disconnected communication channels. Such methods make it difficult to obtain a current view of blood availability and can increase the time required to process a request.

The developed system uses HTML5, CSS3, and Vanilla JavaScript for the browser-based presentation layer. The client communicates through JSON-based HTTP requests with a Node.js and Express.js REST backend. MySQL is used as the relational persistence layer, while the mysql2 package provides database connectivity. Environment-specific database parameters are loaded through dotenv and Cross-Origin Resource Sharing is enabled through cors.

The application implements modules for user registration and login, donor management, blood bank management, inventory management, blood availability search, blood request processing, blood issue recording, dashboard statistics, and contact message submission. The request approval workflow checks the required blood group and available units before reducing inventory and marking a request as approved. This creates a direct relationship between operational request handling and stored inventory.

For cloud deployment, the application is hosted on an AWS EC2 instance running Amazon Linux 2023. Git and GitHub support source-code transfer and version control. The Node.js backend is managed using PM2 so that the API can continue running independently of the SSH terminal, while the static frontend is served using the serve package. The project demonstrates the integration of frontend engineering, REST APIs, relational database design, and EC2-based application hosting in a practical healthcare support system.

TABLE OF CONTENTS

S.No	Content	Page
1	Introduction	5
2	Project Planning and Methodology	7
3	Technology Stack	9
4	System Design and Architecture	11
5	Database and API Design	14
6	Implementation	17
7	AWS EC2 Deployment	20
8	Testing and Results	23
9	Conclusion and Future Scope	25

LIST OF ABBREVIATIONS

Abbreviation	Expanded Form
API	Application Programming Interface
AWS	Amazon Web Services
CORS	Cross-Origin Resource Sharing
CSS	Cascading Style Sheets
EC2	Elastic Compute Cloud
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
JSON	JavaScript Object Notation
PM2	Process Manager 2
REST	Representational State Transfer
SQL	Structured Query Language
UI	User Interface

CHAPTER 1: INTRODUCTION

1.1 Project Introduction

Blood availability is a time-sensitive operational concern. A blood management platform must maintain reliable information about donors, blood banks, stock units, patient requests, and issue history. The Blood Bank Management System developed in this project provides a centralized web interface for these activities and connects the user interface to a persistent MySQL database through a REST backend.

1.2 Project Background

Manual or semi-digital workflows often separate donor information from inventory and request records. A user may know that a blood bank exists but may not have a direct way to inspect blood-group availability. Similarly, administrators may process requests without a consolidated dashboard. The project addresses these coordination gaps by placing major blood bank operations within one browser-based application.

1.3 Existing Problems

- Fragmented donor, blood bank, inventory, and request records.
- Difficulty obtaining a consolidated view of available blood units.
- Manual request processing that can be disconnected from inventory status.
- Limited visibility into issued blood records and operational statistics.
- Dependence on local execution when an application is not deployed to cloud infrastructure.

1.4 Problem Statement

To design, implement, and deploy a web-based Blood Bank Management System that centralizes blood-related records, exposes structured REST endpoints, persists operational data in MySQL, supports blood availability and request workflows, and remains accessible through an AWS EC2 deployment.

1.5 Project Objectives

- Develop a responsive multi-page web interface using HTML5, CSS3, and Vanilla JavaScript.
- Create REST APIs with Node.js and Express.js for all major application modules.
- Maintain structured relational records using MySQL.
- Support donor creation, retrieval, modification, and deletion.
- Maintain blood bank and blood inventory information.
- Allow users to search inventory by blood group and city.
- Record blood requests and provide approve/reject operations.
- Prevent request approval when sufficient blood units are not available.
- Record blood issue transactions and provide dashboard statistics.
- Deploy the application on AWS EC2 Amazon Linux 2023 and keep the backend active with PM2.

1.6 Scope of the System

The project scope covers the software workflow from browser interaction to database persistence and cloud hosting. It includes user registration and login, donor records, blood bank records, inventory records, blood availability display, request processing, issue records, dashboard statistics, and contact messages. The system is designed as a practical academic full-stack project rather than a certified hospital information system.

1.7 Core Project Contributions

The main engineering contribution is the integration of independent frontend pages with a modular Express route layer and a relational MySQL schema. The frontend dynamically builds its API base address from the current browser protocol and hostname and targets port 5000. This allows the same client code to communicate with the backend when hosted on the EC2 machine, without hardcoding a specific public IP address in every JavaScript module.

CHAPTER 2: PROJECT PLANNING AND METHODOLOGY

2.1 Development Methodology

The project followed an iterative module-oriented development process. Instead of implementing the complete system as a single block, major functional areas were developed around database tables and REST route modules. The frontend pages were then connected to the corresponding API endpoints using the Fetch API.

2.2 Requirement Gathering

The initial requirements were derived from common blood bank operations: donor registration, blood bank maintenance, inventory visibility, patient blood requests, approval decisions, and issue records. Cloud deployment requirements were added so that the project could demonstrate AWS infrastructure usage in addition to local full-stack development.

2.3 Functional Requirements

ID	Module	Requirement
FR-01	Authentication	Register users and validate login credentials.
FR-02	Donors	Add, list, update, and delete donor records.
FR-03	Blood Banks	Add, list, and delete blood bank records.
FR-04	Inventory	Store blood group, units, expiry date, and bank.
FR-05	Availability	Display and filter available blood units.
FR-06	Requests	Submit, list, approve, reject, and delete requests.
FR-07	Issues	Record and list blood issue transactions.
FR-08	Dashboard	Return consolidated operational counts.
FR-09	Contact	Persist contact form messages.

2.4 Non-Functional Requirements

Category	Requirement
Usability	Pages should provide clear forms, tables, and navigation.
Maintainability	Backend functions should be separated into route modules.
Portability	Environment values should be externalized using dotenv.
Availability	Production backend should continue after SSH logout using PM2.
Interoperability	Frontend and backend should exchange JSON over HTTP.
Data Integrity	Inventory approval must check sufficient units before deduction.
Scalability	EC2 instance resources can be changed as workload grows.

2.5 Step-by-Step Development Process

- Design the relational database and create the MySQL schema.
- Initialize the Node.js project and install Express, mysql2, dotenv, and cors.
- Create database connection configuration using environment variables.
- Create Express route modules for authentication and operational entities.
- Develop HTML pages and dedicated CSS files for individual modules.
- Connect browser forms and tables to REST APIs using asynchronous fetch calls.
- Test CRUD workflows and the request approval inventory deduction logic.
- Push the project to GitHub and clone the repository on AWS EC2.
- Configure Amazon Linux 2023, Node.js, database environment variables, PM2, and serve.
- Validate the application through the EC2 public endpoint and API port.

2.6 Project Repository Structure

```
Blood Bank Management System/  
├── backend/  
│   ├── config/db.js  
│   ├── routes/  
│   └── server.js  
├── css/  
├── images/  
├── js/  
├── pages/  
├── blood_bank_db.sql  
├── index.html  
└── package.json
```


CHAPTER 3: TECHNOLOGY STACK

3.1 Frontend Technologies

Technology	Purpose
HTML5	Defines semantic structure, forms, tables, navigation, and page content.
CSS3	Provides page-specific styling and common visual presentation.
Vanilla JavaScript	Handles DOM updates, form events, Fetch API calls, filtering, and session storage.

3.2 Backend Technologies

Technology	Purpose
Node.js	Executes the JavaScript backend runtime.
Express.js 5.2.1	Defines the HTTP server, middleware, and REST routes.
mysql2 3.14.3	Connects Node.js to MySQL and executes parameterized queries.
dotenv 17.4.2	Loads database and runtime values from backend/.env.
cors 2.8.5	Enables cross-origin requests between the served client and API.
nodemon 3.1.10	Supports automatic backend restart during development.

3.3 Database Technology

MySQL is used as the relational database. The supplied SQL dump defines seven tables: users, donors, blood_banks, blood_inventory, blood_requests, blood_issues, and contact_messages. The design uses auto-incrementing primary keys and structured columns for operational data.

3.4 Cloud and Deployment Technologies

Technology	Deployment Role
AWS EC2	Virtual server used to host the project.
Amazon Linux 2023	Operating system used by the EC2 instance.
PM2	Keeps the Node.js backend process running and supports restart management.
serve	Hosts the static HTML, CSS, JavaScript, and image files.
Git	Tracks source code and supports repository operations.
GitHub	Remote repository used to transfer the project to EC2.

3.5 Technology Selection Rationale

The selected stack is intentionally lightweight. Vanilla JavaScript avoids a frontend framework build pipeline, while Express provides a direct route-oriented backend. MySQL fits the structured relationships among blood banks, inventory records, requests, and users. EC2 provides operating-system-level control, making it suitable for demonstrating Linux commands, Node.js process management, port configuration, and application deployment.

3.6 Package Configuration

```
{
  "name": "blood-bank-management-system",
  "version": "1.0.0",
  "description": "Blood Bank Management System",
  "main": "backend/server.js",
  "scripts": {
    "start": "node backend/server.js",
    "dev": "nodemon backend/server.js",
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [
    "blood-bank",
    "nodejs",
    "express",
    "mysql"
  ],
  "author": "",
  "license": "ISC",
  "type": "commonjs",
  "dependencies": {
    "cors": "^2.8.5",
    "dotenv": "^17.4.2",
    "express": "^5.2.1",
    "mysql2": "^3.14.3"
  },
  "devDependencies": {
    "nodemon": "^3.1.10"
  }
}
```

The package file identifies `backend/server.js` as the main entry point. The start script executes the backend with Node.js, while the development script uses nodemon.

CHAPTER 4: SYSTEM DESIGN AND ARCHITECTURE

4.1 Overall Architecture

The system follows a three-layer web architecture. The browser presentation layer contains HTML pages, CSS stylesheets, and JavaScript modules. The application layer is a Node.js and Express server that exposes REST routes. The data layer is MySQL, accessed through mysql2. In production, these components are hosted from an AWS EC2 Amazon Linux 2023 environment.

Blood Bank Management System - Deployment Architecture

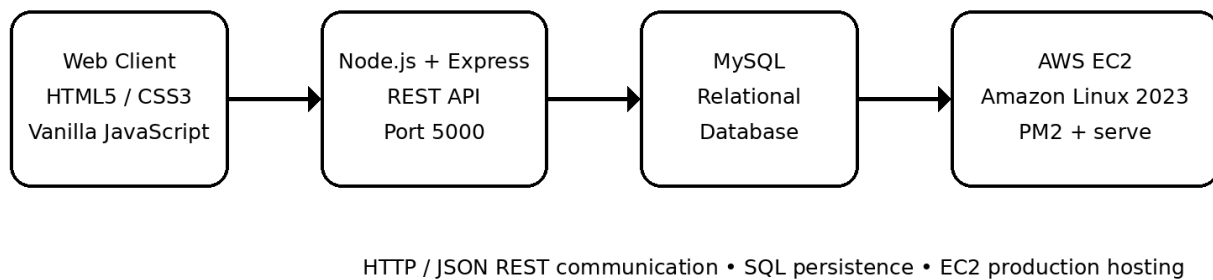


Figure 4.1: Blood Bank Management System deployment architecture

4.2 Frontend Communication Design

Each main JavaScript module defines `API_BASE` using the browser's current protocol and hostname and appends port 5000. Therefore, when the frontend is opened through the EC2 public host, API calls target the same host on the backend port. The Fetch API sends and receives JSON for create, read, update, delete, approve, and reject operations.

4.3 Backend Route Architecture

Base Route	Route Module	Responsibility
/api/auth	authRoutes.js	Registration and login
/api/donors	donorRoutes.js	Donor CRUD operations
/api/bloodbanks	bloodBankRoutes.js	Blood bank maintenance
/api/inventory	inventoryRoutes.js	Inventory storage and joined listing
/api/bloodrequests	bloodRequestRoutes.js	Request lifecycle and approval
/api/issues	bloodIssueRoutes.js	Blood issue records
/api/dashboard	dashboardRoutes.js	Aggregate statistics
/api/contact	contactRoutes.js	Contact message persistence

4.4 Server Initialization Flow

```
require("dotenv").config({ path: __dirname + "/.env" });
const express = require("express");
const cors = require("cors");
const app = express();
```

```
require("./config/db");
app.use(cors());
app.use(express.json());
```

```
app.use("/api/auth", authRoutes);
app.use("/api/donors", donorRoutes);
app.use("/api/bloodbanks", bloodBankRoutes);
app.use("/api/inventory", inventoryRoutes);
app.use("/api/bloodrequests", bloodRequestRoutes);
app.use("/api/issues", bloodIssueRoutes);
app.use("/api/dashboard", dashboardRoutes);
app.use("/api/contact", contactRoutes);
```

The server loads environment configuration, initializes the database connection, enables CORS and JSON parsing, and mounts each route module under a dedicated API prefix.

4.5 Major Application Workflows

4.5.1 Donor Management Workflow

The donor page collects full name, age, gender, blood group, phone, email, city, address, and last donation date. A POST request creates a donor. Existing records are loaded with GET, modified with PUT `/:id`, and removed with DELETE `/:id`.

4.5.2 Blood Availability Workflow

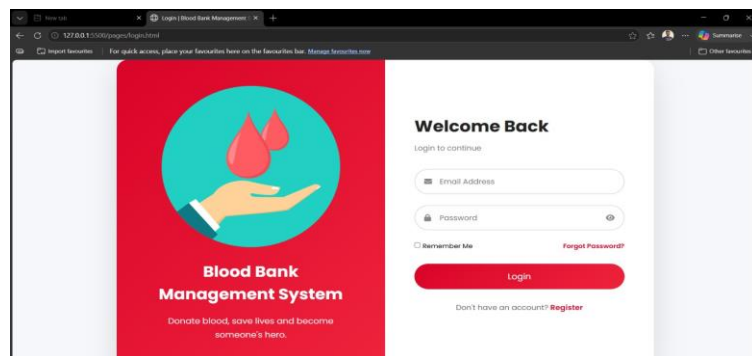
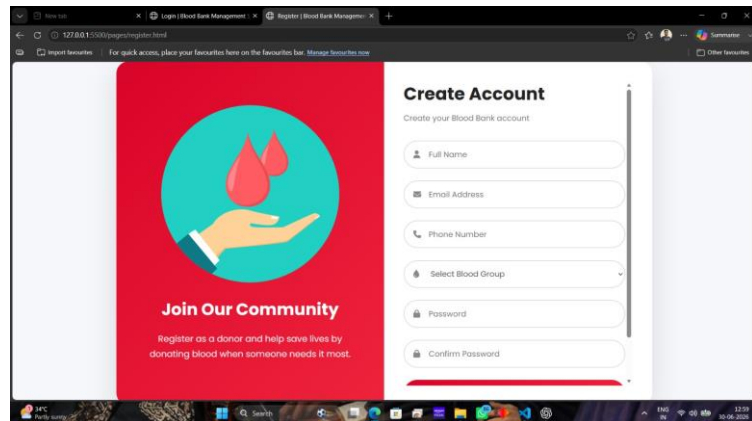
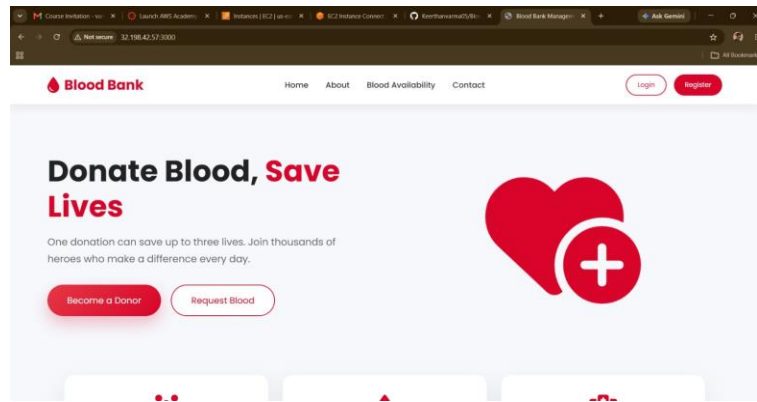
The availability page retrieves joined inventory information from the backend. It can accept a blood group through the page query string and filters the loaded inventory. Users can also search by blood group and city. Results are rendered as blood cards showing group, bank name, city, and available units.

4.5.3 Blood Request Approval Workflow

The approval route first reads the requested blood group and units from `blood_requests`. It then executes an inventory UPDATE that subtracts the required units only where the matching blood group has at least the required quantity. The query uses LIMIT 1. If no row is affected, the API returns 'Insufficient Blood Units'. If inventory is updated, the request status is changed to Approved.

4.5.4 Dashboard Workflow

The dashboard endpoint executes a single SELECT statement containing subqueries for total donors, total blood banks, total available units, total requests, pending requests, and total issues. The browser reads the JSON response and updates dashboard values.



CHAPTER 5: DATABASE AND API DESIGN

5.1 Relational Database Schema

Table	Primary Key	Purpose
users	user_id	Stores registered users and roles.
donors	donor_id	Stores donor personal and donation details.
blood_banks	bank_id	Stores blood bank and manager details.
blood_inventory	inventory_id	Stores blood group units by blood bank.
blood_requests	request_id	Stores patient blood requirements and status.
blood_issues	issue_id	Stores issued blood transaction details.
contact_messages	message_id	Stores contact form submissions.

5.2 Key Relationships

The blood_inventory table is linked operationally to blood_banks through bank_id. The inventory listing route performs an INNER JOIN between these tables and returns the blood bank name and city together with blood group, units available, and expiry date. Blood requests are processed against inventory by matching blood_group.

5.3 Blood Request Status Model

The blood_requests table defines status as an ENUM with Pending, Approved, Rejected, and Completed values, with Pending as the default. The current API implements explicit approval and rejection operations. Approval also performs an inventory quantity check and deduction.

5.4 Database Connection Configuration

```
require("dotenv").config({ path: __dirname + "../.env" });

const mysql = require("mysql2");

const connection = mysql.createConnection({
  host: process.env.DB_HOST,
  user: process.env.DB_USER,
  password: process.env.DB_PASSWORD,
  database: process.env.DB_NAME
});

connection.connect((err) => {
  if (err) {
    console.error("Database Connection Failed");
    console.error(err);
    return;
  }
  console.log("MySQL Connected Successfully");
});

module.exports = connection;
```

Database credentials are not hardcoded in `db.js`. The module reads `DB_HOST`, `DB_USER`, `DB_PASSWORD`, and `DB_NAME` from `backend/.env`. The repository's `.gitignore` excludes `backend/.env`, reducing the risk of accidentally committing environment-specific database credentials.

5.5 SQL Query Safety

The route modules generally use placeholder parameters represented by question marks and provide values separately to `db.query`. This parameterized query pattern is preferable to directly concatenating form values into SQL strings.

5.6 Data Integrity Observation

The approval workflow performs inventory deduction before updating the request status. For stronger production-grade consistency, these two database operations should be wrapped in a MySQL transaction so that both operations either succeed together or roll back together.

5.7 REST API Endpoint Specification

Method	Endpoint	Operation
POST	/api/auth/register	Register user
POST	/api/auth/login	Validate login
GET/POST	/api/donors	List or add donors
PUT/DELETE	/api/donors/:id	Update or delete donor
GET/POST	/api/bloodbanks	List or add blood banks
DELETE	/api/bloodbanks/:id	Delete bank and related inventory
GET/POST	/api/inventory	List or add inventory
DELETE	/api/inventory/:id	Delete inventory record
GET/POST	/api/bloodrequests	List or submit requests
PUT	/api/bloodrequests/approve/:id	Approve and deduct inventory
PUT	/api/bloodrequests/reject/:id	Reject request
DELETE	/api/bloodrequests/:id	Delete request
GET/POST	/api/issues	List or add issue record
DELETE	/api/issues/:id	Delete issue record
GET	/api/dashboard	Get aggregate statistics
POST	/api/contact	Save contact message

5.8 JSON Communication

The frontend sets Content-Type to application/json for POST and PUT requests and serializes JavaScript objects using JSON.stringify. Express parses the request body through express.json(). API responses are returned as JSON objects or result arrays.

CHAPTER 6: IMPLEMENTATION

6.1 User Registration and Login

The registration interface validates that password and confirm-password values match before calling `/api/auth/register`. The backend inserts `full_name`, `email`, `password`, and `role` into `users`. The client sends the `role` value `Donor`. During login, the API searches for a row matching email and password. On success, the browser stores the returned user object in `sessionStorage` and redirects to the home page.

6.2 Donor Module

The donor JavaScript module maintains an `editId` variable. New submissions use `POST`. When an existing donor is selected for editing, `editId` is assigned and the same form submits a `PUT` request to `/api/donors/:id`. After completion, the form is reset and the donor list is reloaded.

6.3 Blood Bank Module

Blood bank records contain bank name, manager name, email, phone, city, capacity, and address. The backend supports creation and listing. During deletion, inventory rows for the selected bank are deleted before the blood bank row itself is removed.

6.4 Inventory Module

The inventory page first loads blood banks to populate a selection control. An inventory record contains `bank_id`, `blood_group`, `units_available`, and `expiry_date`. Listing uses a database join so the user sees the bank name and city instead of only the numeric bank identifier.

6.5 Blood Request Implementation

```
UPDATE blood_inventory
SET units_available = units_available - ?
WHERE blood_group = ?
AND units_available >= ?
LIMIT 1
```

The request approval logic is the most important operational rule in the current project. The query only subtracts units when the selected inventory row contains the requested blood group and has enough units. The `affectedRows` value is checked to determine whether inventory was updated.

If `affectedRows` is zero, the backend returns a failure response with the message 'Insufficient Blood Units'. Otherwise, the request is marked `Approved`. The browser disables approval and rejection controls when a request is no longer `Pending`.

6.6 Blood Issue Module

The issue form collects patient name, hospital name, blood group, units issued, issue date, and issued-by information. Client-side checks prevent submission when required fields are empty. The backend stores the issue record and supports listing and deletion.

6.7 Contact Module

The contact page captures full name, email, subject, and message. The JavaScript module submits these values to `/api/contact`, and the backend inserts them into `contact_messages`.

6.8 Frontend API Address Strategy

```
const API_BASE =  
  `${window.location.protocol}//${window.location.hostname}:5000`;
```

This expression appears across the project's JavaScript modules. It derives the backend host from the host currently serving the frontend. During EC2 deployment, if the frontend is opened through the EC2 public host, requests are automatically directed to the same public host on port 5000.

6.9 Project Page Modules

Page	Purpose
index.html	Home page and dashboard presentation
login.html / register.html	User access interfaces
donor.html	Donor management
blood-bank.html	Blood bank management
inventory.html	Blood inventory management
blood-availability.html	Availability search and display
blood-request.html	Request submission and processing
blood-issue.html	Issue transaction records
contact.html	Contact message submission
about.html	Project information

6.10 Current Security Limitation

The current authentication route compares passwords directly with stored password values and returns the complete matching user row. This is acceptable only as an academic prototype. A production deployment should hash passwords with bcrypt or Argon2, issue secure sessions or JWTs, validate roles on protected routes, restrict CORS origins, and never return password fields to the browser.

CHAPTER 7: AWS EC2 DEPLOYMENT

7.1 Deployment Overview

The project is deployed on an Amazon EC2 virtual machine using Amazon Linux 2023. EC2 provides a Linux server on which the project repository, Node.js runtime, backend process manager, frontend static server, and environment configuration can be maintained. The deployment model used for this project runs the Express API on port 5000 and serves the static frontend separately using serve.

7.2 EC2 Instance Preparation

- Launch an EC2 instance with Amazon Linux 2023.
- Create or select an EC2 key pair and securely store the private key.
- Configure the security group for SSH administration and the application ports required by the project.
- Connect to the instance using SSH.
- Update installed packages before application setup.

```
sudo dnf update -y
sudo dnf install git -y
git --version
```

7.3 Security Group Considerations

SSH port 22 should be restricted to the administrator's trusted IP address wherever possible. The frontend serving port and backend port 5000 must be reachable according to the deployment design. For a stronger production design, a reverse proxy such as Nginx should expose only HTTP/HTTPS and forward API traffic internally, avoiding direct public exposure of port 5000.

7.4 Node.js and Project Setup

After connecting to the EC2 instance, Node.js and npm are installed. The project repository is then cloned from GitHub. The supplied repository configuration identifies the Blood-Bank-Management-System GitHub repository as the origin.

```
# Verify Node.js and npm after installation
node -v
npm -v

# Clone project source
git clone https://github.com/Keerthanvarma05/Blood-Bank-Management-System.git
cd Blood-Bank-Management-System

# Install project dependencies
npm install
```

7.5 MySQL Environment Configuration

The backend expects four database environment variables. The backend/.env file is intentionally excluded from Git. Therefore, it must be created securely on the EC2 host or populated through an equivalent secret-management approach.

```
DB_HOST=<mysql-host>
DB_USER=<mysql-user>
DB_PASSWORD=<mysql-password>
DB_NAME=<database-name>
PORT=5000
```

The MySQL schema is initialized from blood_bank_db.sql. The database must be reachable from the EC2 instance. Database network rules should permit only required sources, and a dedicated database user should be granted only the permissions needed by the application.

7.6 Backend Process Management with PM2

Running `node backend/server.js` directly ties the server process to the active terminal session. PM2 is used to keep the Express backend running after SSH disconnection and to simplify process status and restart operations.

```
sudo npm install -g pm2

pm2 start backend/server.js --name bbms-backend
pm2 status
pm2 logs bbms-backend

# Configure process restoration after reboot
pm2 startup
# Run the command printed by PM2, then:
pm2 save
```

7.7 Serving the Static Frontend

The project frontend consists of static HTML, CSS, JavaScript, SVG, and image resources. The `serve` package can host the project directory as static content.

```
sudo npm install -g serve

# From the project root
serve -s . -l 3000
```

The frontend can be kept under a process manager as well. When the browser opens the frontend from the EC2 host, the `API_BASE` expression derives the same hostname and sends backend requests to port 5000.

7.8 Deployment Verification

```
pm2 status
pm2 logs bbms-backend
curl http://localhost:5000/
curl http://localhost:5000/api/dashboard
```

The backend root route should return 'Blood Bank Backend Running Successfully'. The dashboard endpoint should return a JSON object containing aggregate system statistics.

CHAPTER 8: TESTING AND RESULTS

8.1 Functional Test Cases

Test	Input / Action	Expected Result
TC-01	Register valid user	Registration success response
TC-02	Login with matching credentials	Login success and user session stored
TC-03	Add donor	Donor inserted and displayed
TC-04	Edit donor	Selected donor record updated
TC-05	Add blood bank	Blood bank displayed in list
TC-06	Add inventory	Inventory linked with selected bank
TC-07	Search availability	Matching blood cards displayed
TC-08	Submit blood request	Pending request created
TC-09	Approve with sufficient units	Units deducted and request approved
TC-10	Approve with insufficient units	Failure message returned
TC-11	Reject pending request	Status changed to Rejected
TC-12	Add blood issue	Issue record stored and displayed
TC-13	Open dashboard	Aggregate counts returned
TC-14	Submit contact message	Message persisted

8.2 Integration Testing

Integration testing focuses on the complete path from an HTML form to JavaScript, Fetch API, Express route, mysql2 query, and MySQL result. The inventory listing additionally validates the INNER JOIN between blood_inventory and blood_banks. Request approval validates the dependency between blood_requests and blood_inventory.

8.3 Deployment and Operational Results

The EC2 deployment demonstrates that the application can move from a local development environment to a Linux cloud server. GitHub provides the source transfer path, npm installs the declared dependencies, dotenv supplies host-specific configuration, PM2 maintains the backend process, and serve publishes the static frontend.

8.4 Result Summary

Area	Observed Project Capability
Authentication	Registration and credential-based login implemented.
Donor Records	Create, read, update, and delete operations implemented.
Blood Banks	Create, list, and delete operations implemented.
Inventory	Bank-linked inventory and availability listing implemented.
Requests	Submission, approval, rejection, and deletion implemented.
Inventory Rule	Approval checks units before deduction.
Issue Records	Create, list, and delete operations implemented.
Dashboard	Aggregate operational counts implemented.
Cloud Hosting	EC2 Amazon Linux 2023 deployment workflow documented.
Process Management	PM2 used for persistent Node.js backend operation.

8.5 Limitations Identified During Review

- Passwords are currently stored and compared without cryptographic hashing.
- Authentication does not yet enforce authorization middleware on administrative APIs.
- Request approval and inventory deduction should use a database transaction.
- The issue-record creation route does not automatically deduct inventory.
- Direct exposure of port 5000 is weaker than an HTTPS reverse-proxy deployment.
- Automated tests are not configured; package.json currently contains a placeholder test script.

CHAPTER 9: CONCLUSION AND FUTURE SCOPE

9.1 Conclusion

The Blood Bank Management System successfully demonstrates the development of a modular full-stack web application using HTML5, CSS3, Vanilla JavaScript, Node.js, Express.js, and MySQL. The project centralizes donor information, blood bank records, inventory, availability search, patient blood requests, issue records, dashboard statistics, and contact messages.

The backend is divided into route modules and communicates with MySQL through mysql2. The browser uses asynchronous Fetch API requests and dynamically targets the backend host. The request approval logic provides an important operational rule by checking blood availability before deducting units and approving a request.

Deployment on AWS EC2 with Amazon Linux 2023 extends the project beyond local development. Git and GitHub support source management, dotenv separates environment configuration, PM2 maintains the backend process, and serve hosts the static frontend. Together, these technologies demonstrate a complete development-to-cloud workflow.

9.2 Future Enhancements

- Hash passwords using bcrypt or Argon2 and implement JWT/session authentication.
- Add role-based authorization for admin, blood bank staff, donor, and requester operations.
- Use MySQL transactions and row locking for atomic request approval.
- Automatically deduct and reconcile inventory when blood is issued.
- Add expiry alerts and prevent issue of expired blood units.
- Add donor eligibility checks and donation interval validation.
- Deploy Nginx with HTTPS and route /api traffic internally to Express.
- Move database credentials to AWS Secrets Manager or Systems Manager Parameter Store.
- Add automated API tests, health checks, centralized logs, and CloudWatch monitoring.
- Add email/SMS notifications for request status and urgent blood requirements.

9.3 Final Statement

The current implementation is a strong academic full-stack and AWS deployment project. Its next engineering priority should be security and transactional correctness rather than adding more visual features. Strengthening authentication, authorization, atomic inventory updates, and HTTPS deployment would move the system significantly closer to a production-oriented architecture.